

Why

Tipsoi as an AI-Native Attendance Ecosystem

Contents	
Tab 1	Why
Tab 2	Approach 1 — the phased build (V1, V2, V3)
Tab 3	Approach 2 — the modular agent architecture
Tab 4	Comparison — pros, cons, cost, and dev time

Why

1. The Moment We Are In

"We are only months into the Agentic AI era. Just as early electricity had poor initial ROI, current AI usage is a foundational investment — not a cost to be minimised."
— Pedro Franceschi, CEO of Brex

Every category of software is being rethought for Industry 4.0 right now. HRMS, ERP, shift management, and attendance systems are not immune. The question is not whether AI will change how these products work — it already is. The question is whether Tipsoi leads that change in our market or follows someone who did.

The companies winning with AI are *not* the ones adding a chatbot to their UI. They are the ones asking: "If we were building this product from scratch today, knowing what AI can do, what would we build?"

This note is that question, applied to Tipsoi.

2. The Core Idea

Today, getting value out of Tipsoi requires knowing where to go. You navigate to the right module, configure the right filters, select a date range, and generate a report. It works — but only if you already know what you want and how to get it.

The agentic model flips this. The user says what they want, in plain language. The system figures out the rest.

Example

"Generate an overtime report for employees who worked more than 20 hours this month, broken down by department. Schedule it every Monday and email it to the factory managers."

That is not a chatbot. That is an agent: it understands the intent, retrieves the data, runs the calculations, assembles a report, and sets up a recurring job — all from one sentence.

This is what Pedro Franceschi at Brex calls "freeing the claw" — letting AI agents operate in loops with real tools, rather than just answering questions.

3. What We Could Actually Build

The following are not hypothetical. These are capabilities that are buildable today, on top of Tipsoi's existing data, using current LLM technology.

Report Generation by Conversation

Instead of navigating report modules, an HR Manager types what they need. The system calls the right Tipsoi report API, retrieves the data, and returns a formatted output — table, chart, or PDF — in seconds. The user approves before anything is saved or sent.

- "Show attendance for night shift security staff in the Chattogram facility, last 30 days"
- "Compare overtime costs between our two factories for Q1 vs Q2"
- "Which employees were late more than three times this month?"

Scheduled Insight Delivery

Once a report is approved, the user can save it and schedule it. The system reruns the same logic automatically — daily, weekly, monthly — and delivers results to email or the notification centre. This eliminates a class of recurring manual work entirely.

Proactive Anomaly Alerts

Rather than waiting to be asked, the system surfaces things worth knowing: a department with unusually high absenteeism this week, an attendance pattern that suggests attrition risk, an overtime spike worth reviewing. HR gets insights they did not know to look for.

Domain-Specific Virtual Employees

The longer-term vision — articulated by Brex as "Company AGI" — is agents that act as specialised teammates. An Attendance Agent that understands your shift configurations and flags exceptions. A future Payroll Agent that knows your pay cycles and deduction rules. Not a generic assistant, but one grounded in the context of a specific client's operation.

4. How It Would Work (Simply)

	Stage	What Happens
1	User speaks	Plain language request entered in a chat interface inside Tipsoi
2	Intent resolved	AI classifies the domain, filters, time range, and output type needed
3	Data retrieved	System calls the relevant existing Tipsoi report API (read-only)
4	Output assembled	Charts, tables, and a narrative summary are generated for review
5	Human approves	User reviews and confirms before any report is saved, sent, or scheduled
6	Optionally scheduled	Approved configuration saved for automatic re-execution on a cadence

Non-negotiable guardrail
The AI never modifies data. It calls read-only report APIs only. Every output that touches payroll, attendance records, or employee status requires an explicit human approval step before any action. This is the architectural principle that makes the whole system safe to deploy.

5. Three Paths Forward

We have a strategic choice to make. Each path has a different risk profile and upside.

	Path A: Wait & Watch	Path B: AI Layer on Tipsoi	Path C: AI-Native Rebuild
What it is	Monitor the market. Add AI when clients demand it.	Build an agentic layer on top of the existing platform now.	Redesign Tipsoi from scratch around AI as the primary interface.
Upside	Low risk, no investment.	Significant UX improvement. Differentiated. Buildable in months.	Maximum long-term moat.
Risk	Cede the window to a faster competitor.	Requires LLM investment and disciplined execution.	High cost, high risk, long timeline.
Verdict	Not viable. The window is open now.	Recommended. High ROI, contained scope.	Not yet. Revisit in 12–18 months.

Path B is the recommendation. It does not require rebuilding Tipsoi. It adds an agentic interface layer on top of the existing database and modules. The existing product continues to work exactly as it does today — this is additive, not a replacement.

This dossier presents two ways to execute Path B. They are detailed in Tab 2 (a phased build) and Tab 3 (a modular architecture), and compared directly in Tab 4.

6. Assessment

Why We Should Do This	What We Need to Get Right
Non-technical HR staff can finally self-serve on analytics	LLM outputs must always pass through human review before action
Recurring manual report work gets fully automated	Data sent to third-party LLM APIs needs a privacy/legal review
Meaningful differentiator in Bangladesh and international markets	Agents must call read-only APIs only — never write paths
Reduces support load — fewer 'how do I run this report' tickets	Latency of 2–5 seconds per AI query needs a streaming UI to feel acceptable
Client onboarding gets faster — new orgs extract value immediately	LLM API costs scale with usage — needs per-client cost monitoring from day one
Positions Tipsoi for the next generation of HR software buyers	Bangla language quality varies across providers — needs evaluation

7. What We Are Asking For

This is a brainstorm, not a project plan. We are asking for one thing at this stage:

The Ask

To run a 6–8 week prototype with one pilot client — covering Attendance, Leave, and Overtime queries built on Tipsoi's existing report APIs. Outcome: a validated proof of concept, cost benchmarks, and a decision on whether to proceed further. (Payroll is deliberately excluded from the prototype because the payroll API is not yet exposed — see Tab 2.)

If the prototype does not meet our accuracy and latency bar, we stop. If it does, we have the evidence to move quickly.

"Always start with the problem. Ask: why can't I solve this with AI? The bottleneck is no longer what AI can do — it is choosing the right problems to point it at."

— Pedro Franceschi, CEO of Brex

The problem is clear. Our clients spend too much time navigating a system to extract information they already know exists. AI can close that gap. The only question is how effortless we can make it.

Approach 1

APPROACH 1

The Phased Build

Agentic AI on Tipsoi, shipped as versioned releases — what we can build right now, and what comes later.

1. What the API Already Gives Us

The Tipsoi API is more capable than it might appear from the UI. Before writing a single line of AI code, we already have the building blocks for a meaningful first release. The API exposes nine report endpoints, fully parameterised and ready to query.

API Endpoint	What It Returns	Key Filters
Monthly Attendance	Full month attendance per employee	officelId, from/to, employeeName
Daily Summary	Day-level punch summary, all employees	officelId, from/to, reportType
Daily Absent Report	Employees absent on a day, by type	officelId, from/to, absentType
Late Report	Employees arriving late in a range	officelId, from/to, employeeName
Leave History	Leave taken per employee in a range	officelId, from/to, status
Leave Balance	Remaining leave quota per employee	officelId, from/to
Mobile Punch Report	Punch-ins from mobile	officelId, from/to, lunch/dinner
Monthly OT Report	Monthly overtime hours per employee	officelId, companyId, from/to
Daily OT Report	Day-level overtime breakdown	officelId, companyId, from/to

Supporting data is also available: employee list (with status, department, designation), shift and shift group list, department list, workplace list, and leave policy config. This is enough context for an agent to understand the org structure when answering any query.

The Insight

We do not need to touch the database directly. The existing report APIs are the data layer. The AI's job is to understand what the user wants, map it to the right API call with the right parameters, fetch the result, and present it clearly.

2. The Low-Hanging Fruits

These are the use cases we can build first — using only existing APIs, requiring no new backend work. Ordered by impact and ease.

#	What the User Says	What the Agent Does	API Used	Effort
—	ATTENDANCE			
1	"Who was absent today?"	Calls Daily Absent Report for today, grouped by department.	Daily Absent	Low
2	"Late arrivals this week for Dhaka office"	Resolves office ID, sets week range, calls Late Report.	Late Report	Low
3	"Monthly attendance summary for May"	Sets May timestamps, calls Monthly Attendance.	Monthly Att.	Low
4	"Who punched in from mobile this month?"	Calls Mobile Punch Report, surfaces list with location.	Mobile Punch	Low
5	"Absenteeism trend for security, 3 months"	Calls Daily Absent across 3 ranges, aggregates by dept.	Daily Absent (×3)	Med
—	LEAVE			
6	"Who is on leave this week?"	Calls Leave History, current week, approved filter.	Leave History	Low
7	"How much leave balance does Rahim have?"	Resolves employee ID, calls Leave Balance.	Leave Bal. + Emp.	Low
8	"Show all pending leave requests"	Calls Applied Leave List, status=0 (pending).	Leave App.	Low
—	OVERTIME			
9	"Who worked more than 20 OT hours this month?"	Calls Monthly OT, filters > 20, ranks list.	Monthly OT	Low
10	"Compare OT between Factory A and B for Q2"	Calls Monthly OT twice, side-by-side compare.	Monthly OT (×2)	Med
11	"Daily OT breakdown for last week"	Calls Daily OT for past 7 days.	Daily OT	Low
—	CROSS-DOMAIN (still in scope)			
12	"Late AND on overtime this month"	Calls Late + Monthly OT, intersects employee IDs.	Late + OT	Med
13	">5 leave days AND low attendance"	Calls Leave History + Monthly Att., cross-references.	Leave + Att.	Med

Low effort = single API call + parameter mapping. Medium = multiple API calls + result merging. All achievable without any new backend endpoints.

3. What the Agent Does (Under the Hood)

	Step	Detail
1	Parse Intent	Identify report type, date range, office/department filter, employee name, any threshold.
2	Resolve Context	Look up officeld from workplace list. Resolve employee name to ID. Translate relative dates to Unix timestamps.
3	Call API(s)	Construct GET request(s) with correct parameters. Handle pagination. Merge results if multiple calls needed.
4	Present Output	Format as table or summary with a plain-language narrative. Offer PDF/Excel export.

Key Point on Timestamps

Tipsoi's APIs use Unix epoch milliseconds for date filtering. The LLM must reliably translate 'this month', 'last week', 'Q2' into the correct epoch range. This is deterministic and testable, not a hallucination risk.

4. What Is Out of Reach Early (and Why)

Being honest about limitations now prevents scope creep and failed promises.

Capability	Why Not Now	When
Payroll report queries	No payroll API endpoints in the current collection.	After payroll API is exposed
Salary anomaly detection	Requires salary data access. Not in API collection.	After payroll API
Proactive alerts / push	Needs a scheduler service to run unattended.	Later release
Writing data (approve, edit)	Read-only first. Write APIs need a full review flow.	Later, with approval UX
Bangla language input	LLM Bangla quality varies. English-first reduces risk.	After provider evaluation
Biometric / device analytics	Out of scope for the report-query use case.	Future

5. What We Actually Need to Build

Component	What It Does	New or Existing
Chat interface	Text input in the Tipsoi admin panel; result shown inline.	New (frontend)
LLM integration	Sends query + context to an LLM with tool calling enabled.	New

Component	What It Does	New or Existing
Context resolver	Fetches employee/workplace/dept lists at session start.	New (lightweight)
API tool wrappers	Each report endpoint wrapped as a typed, LLM-callable tool.	New (one per endpoint)
Result formatter	Converts raw API JSON into readable tables and summaries.	New
Tipsoi Report APIs	The 9 existing report endpoints. No changes needed.	Existing
Auth (JWT)	Existing token auth. Agent calls APIs as the logged-in user.	Existing

RBAC for Free

Because the agent calls existing Tipsoi APIs using the user's own JWT token, role-based access control is inherited automatically. A factory manager can only query their own officeld. An Admin sees everything. No separate permission layer needed.

6. The Phased Roadmap

Minimal surface area first — solve one high-value problem before expanding. Each phase begins only once the previous one is proven in production.

Phase	Name	What Gets Built	Gate to Unlock Next
V1	Report Assistant	Natural language → report draft → human approval. Attendance, Leave, Overtime.	Accuracy validated on the 13 use cases with 1 pilot client
V2	Scheduled Reports + Payroll	Save and auto-run approved reports. Add Payroll once its API is exposed.	10+ scheduled reports running error-free for 30 days
V3	Proactive Insights	Anomaly alerts, trend detection. AI surfaces things unprompted.	Client feedback confirms insights are actionable, not noise
V4	Workflow Agents	Semi-autonomous multi-step workflows with approval checkpoints.	V3 stable; dedicated AI infrastructure in place

Scope Principle

If it can't be answered by one of the 9 existing report APIs, it is out of V1. This is the constraint that keeps the first release buildable in 6–8 weeks.

7. Bottom Line

The API already exists. The data is already there. The report endpoints already filter by date, office, employee, and status. What is missing is the natural language layer that lets a non-technical HR Manager ask for any of this without touching a single filter dropdown.

That is V1. And based on the API collection, it is buildable without a single new backend endpoint.

Approach 2

APPROACH 2

The Modular Agent Architecture

One stable orchestrator. Specialist agents around it. Add, remove, or replace any agent without touching anything else.

The core idea in one sentence

One stable orchestrator at the centre.
Specialist agents around it. Add, remove, or replace any agent without touching anything.

The principle it follows

Just-in-time and Kaizen. Ship the smallest useful thing. Improve it in place. Never rebuild the foundation.

1. Why Not V1, V2, V3

The conventional approach goes: ship a simple chatbot, add features in V2, become agentic in V3. Every phase is a partial rebuild. Teams spend energy migrating, not improving. Users experience capability cliffs between versions rather than a system that quietly gets better.

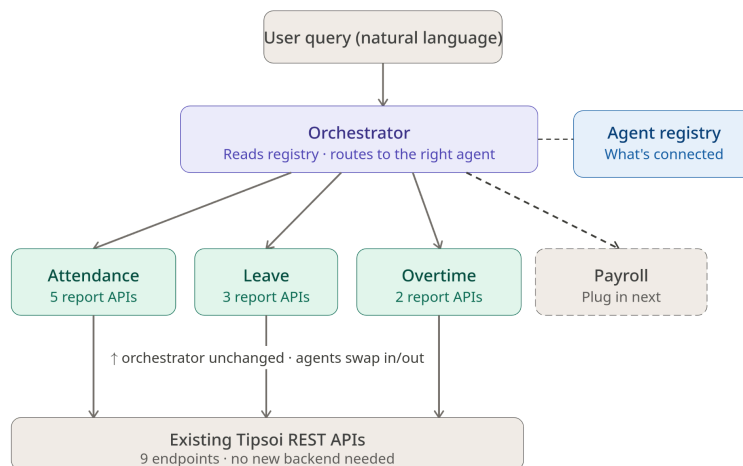
The Kaizen alternative is different. You build the right architecture once — a modular one — and improve it continuously in small, safe, reversible increments. Agents ship when there is demand for them, not in anticipation of it. The orchestrator, once built, never needs to change.

The Shift

Instead of asking 'what do we build in V1?' — ask 'what is the agentic foundation with modular plugin support system we can ship that is extensible enough to never need replacing?'

2. The Architecture

The system has two fixed components and one growing one.



The Orchestrator (fixed — never changes)

A single LLM-powered router. It receives the user's natural language query, reads the agent registry to understand what capabilities are available, selects the appropriate agent(s), and assembles the result. It does not know how any agent works internally — only what each can do, described in plain language in the registry.

This is the hub in a hub-and-spoke model. It routes. It does not compute. It does not know about Tipsoi's database. It knows there is an Attendance Agent that answers attendance questions, and it calls it.

The Agent Registry (fixed structure — entries grow)

A simple lookup table — JSON or database — listing every available agent with three things: its name, a plain-language description of what it handles, and how to call it. Adding an agent means adding one entry. Removing one means deleting one entry. The orchestrator code does not change either way.

```
// Agent registry entry — one per agent
{
  "name": "attendance_agent",
  "handles": "Attendance reports — daily summaries, absent
             employees, late arrivals, monthly attendance",
  "endpoint": "mcp://localhost:8001"
}
```

The Agents (the growing part)

Each agent is a self-contained MCP server — a small Python service that wraps one domain's capabilities behind a standardised interface. The orchestrator calls any agent the same way, regardless of what it does internally. An agent has one responsibility, shares no state with other agents, and can be deployed, updated, restarted, or replaced independently.

Why MCP

Model Context Protocol (MCP) is the interface standard that makes plug-and-play real. Every agent exposes the same kind of interface — tools with typed inputs and outputs. The orchestrator never needs to know how an agent is implemented, only what it can do. MCP is the contract. Agents are the implementations.

3. The Tipsoi Agents

Each agent maps to a domain and wraps the relevant existing REST API endpoints. No new backend is needed. The agents are a thin layer of intelligence on top of APIs that already exist.

Attendance Agent <i>Ship first</i> Daily summaries, absent employees, late arrivals, monthly attendance, and mobile punches. Wraps 5 existing report endpoints.	Leave Agent <i>Ship first</i> Leave balance, leave history, and pending requests. Wraps 3 existing leave endpoints. Cross-references the employee list for name resolution.
Overtime Agent <i>Ship first</i> Monthly and daily overtime reports. Filters by threshold (e.g. > 20 hours). Wraps 2 existing OT endpoints.	Payroll Agent <i>Plug in when API is ready</i> Salary summaries, deduction breakdowns, OT payment queries. Blocked on payroll API availability — not on the orchestrator or other agents.

Each agent is independent. The Payroll Agent being unbuilt has zero effect on the other three. When it is ready, it is registered. When it is not, it simply does not exist in the registry, and the orchestrator routes around its absence gracefully.

4. What an Agent Actually Is (Concretely)

An agent is a Python file. Using FastMCP, it takes about 60 lines to define one. Here is the skeleton:

```
from fastmcp import FastMCP
import httpx

mcp = FastMCP('Attendance Agent')

@mcp.tool()
async def get_late_report(
    office_id: str,
    from_date: int,    # Unix epoch milliseconds
    to_date: int,
    employee_name: str = ''
) -> dict:
    """Returns employees who arrived late in the date range."""
    async with httpx.AsyncClient() as client:
        r = await client.get(
            f'{TIPSOI_BASE}/attendance/leave-late-absent',
            params={'officeId': office_id, 'from': from_date,
                  'to': to_date, 'reportType': 2},
```

```

        headers={'Authorization': f'Bearer {token}'}
    )
    return r.json()

if __name__ == '__main__':
    mcp.run(port=8001)

```

That is the entire agent for one report type. One function, one API call, typed parameters. Add another `@mcp.tool()` for each report endpoint in that domain. Register the server. Done.

Then, we can also use it with Claude/ChatGPT so users can interact with Tipsoi via their own choice of chatbot.

The date problem

Tipsoi APIs use Unix epoch milliseconds for date filtering. The orchestrator's LLM must translate 'last week' and 'Q2' into correct epoch ranges. This is solved with a deterministic `resolve_date_range` tool the LLM calls — not something it computes itself. The highest-frequency failure mode to test first.

5. How a Query Flows Through the System

- 1 **User types a natural language query**
"Who was late more than 3 times this month in the Dhaka office?"
- 2 **Orchestrator reads the agent registry**
Sees Attendance, Leave, Overtime agents. Identifies this as an attendance / late-arrival query.
- 3 **Orchestrator calls `resolve_date_range`**
Translates 'this month' → epoch ms range. Deterministic, not guessed by the LLM.
- 4 **Orchestrator calls `resolve_office`**
Translates 'Dhaka office' → `officeId 4397` from the workplace list, cached at session start.
- 5 **Orchestrator calls the Attendance Agent**
Passes typed parameters. Agent calls the Late Report endpoint, returns structured JSON.
- 6 **Orchestrator formats the result**
Filters `late_count > 3`. Writes a summary: '6 employees were late more than 3 times in June.' Offers table + export.
- 7 **User sees the result and can act**

Approve, export PDF/Excel, or ask a follow-up. Nothing is written without explicit user action.

6. The Kaizen Operating Model

Action	What Actually Happens
Add a new agent	Write the Python file. Register it. Restart orchestrator. No other agents touched.
Remove an agent	Delete the registry entry. The orchestrator stops routing to it. Others unaffected.
Improve an agent	Update its tool functions. Redeploy the single service. Zero downtime on others.
Swap the LLM provider	Change one config line in the orchestrator. Every agent keeps working.
Add a new report type	Add one <code>@mcp.tool()</code> to the relevant agent. The agent already knows about it.
Add Bangla support	Change the orchestrator's LLM prompt. Agents are language-agnostic.
Add scheduling	Add a Scheduler Agent. Orchestrator routes scheduling requests to it. Nothing else changes.

The 5S Principle Applied

Sort: each agent does one thing. Set in order: the registry is the single source of truth. Shine: each agent is inspectable and testable in isolation. Standardise: MCP is the contract for every agent. Sustain: the orchestrator is stable; improvement happens at the agent level.

7. The Immovable Parts

Component	Why It Never Changes
Orchestrator	It has no domain knowledge. It only routes. Adding a new domain means adding an agent — not modifying the orchestrator.
MCP interface contract	Every agent speaks MCP. Any agent can be replaced with a better implementation of the same interface without the orchestrator knowing. It is the wall socket — what you plug in can change; the socket does not.

Everything else — the agents, the registry entries, the API endpoints they call, the LLM provider, the output formatting — can change. Change in one place does not propagate elsewhere.

8. What to Build First

Start with the minimum that demonstrates the full architecture — not the minimum chatbot. The goal of the first build is to prove the system, not just a feature.

	Deliverable	Why This, Not Something Simpler
1	Orchestrator with registry	Proves the routing logic. Adding future agents costs zero orchestrator changes.
2	Three active MCP agents	Attendance, Leave, OT. Three domains prove the multi-agent pattern, not a single hardcoded tool.
3	Date resolution tool	The highest failure risk. Validate exhaustively before anything else.
4	Context resolver	Session-start fetch of employee/workplace/dept lists. Reused by every agent forever.
5	Observability (Logfire)	Every tool call logged with parameters, response, latency. Non-negotiable from day one.
6	Chat UI in Tipsoi admin	Text input that calls the orchestrator API. The UX surface for everything that follows.

That is the prototype. Six components, 6–8 weeks. When it ships, the architecture can already accept a Payroll Agent, a Scheduler Agent, or a Proactive Alerts Agent — without a rewrite.

9. The Non-Negotiable Guardrails

Rule	What It Means in Practice
Agents are read-only	No agent may write, update, or delete any Tipsoi record. Every agent calls GET endpoints only. Write operations stay in the Tipsoi UI with explicit human action.
RBAC is inherited, not rebuilt	Agents call Tipsoi APIs using the logged-in user's JWT. Access control is enforced at the API level automatically. The system gets correct access control for free.

The architecture described here is not a roadmap. It is a system design. The roadmap is whatever the next most valuable agent is — decided by what clients actually ask for, built in a week, registered, shipped.

Comparison

Comparison

The phased build vs the modular architecture — pros, cons, cost, development time, and a recommendation.

1. The Two Approaches at a Glance

Both approaches deliver the same first-release capability: a natural language interface that answers Attendance, Leave, and Overtime questions on top of Tipsoi's existing APIs. They differ in how the system is structured underneath — and that structural difference shapes everything that comes after the first release.

	Approach 1: Phased Build	Approach 2: Modular Architecture
Structure	One LLM with a set of tool wrappers, grown feature by feature.	An orchestrator routing to independent MCP agents.
Growth model	Versioned releases — V1, V2, V3, each adding features.	Continuous — add or remove agents any time, no version cliffs.
Mental model	A product roadmap.	A system that evolves in place.
First release	Same scope: Attendance, Leave, Overtime.	Same scope: Attendance, Leave, Overtime.
Adding payroll later	Extend the monolith with new tools in V2.	Register one new agent. Nothing else changes.

2. Pros and Cons

Approach 1 — The Phased Build

Pros	Cons
Simplest possible thing that works — fastest path to a first demo	Each new domain means extending one growing codebase, which gets harder over time
Lowest initial engineering effort and smallest moving-part count	Version cliffs — V2 and V3 involve partial reworks, not just additions
Easy for a small team to hold entirely in their heads	Tighter coupling — a change in one area risks breaking another
No new infrastructure concepts (MCP, registry) to learn	Harder to let different developers own different domains independently

Approach 2 — The Modular Architecture

Pros	Cons
Add, remove, or replace agents with zero impact on the rest	Higher upfront effort — orchestrator + registry must be built before the first agent pays off
No version cliffs — the system improves continuously (true Kaizen)	More moving parts to deploy and monitor (multiple services, not one)
Different developers can own different agents in parallel	Team must learn MCP and the orchestrator pattern
Each agent is independently testable and debuggable in isolation	Slight latency overhead from the routing layer (usually negligible)
Swapping the LLM provider is a one-line change	Over-engineered if the system never grows beyond a handful of report types

3. Development Time

Both approaches target the same 6–8 week window for a first pilot. The difference is where the effort lands and what you have at the end of it.

Build Phase	Approach 1	Approach 2
Foundation (orchestrator / registry)	Minimal — just an LLM + tool loop (~3–5 days)	Orchestrator + registry + MCP scaffolding (~1.5–2 weeks)
First three domains	Tool wrappers for 9 endpoints (~2 weeks)	Three MCP agents wrapping the same 9 endpoints (~2 weeks)
Shared plumbing (dates, context, formatting)	~1.5 weeks	~1.5 weeks (reused by all agents)
Chat UI + observability + testing	~1.5 weeks	~1.5 weeks
Total to first pilot	~6 weeks	~7–8 weeks
Adding the 4th domain (e.g. Payroll)	~1.5–2 weeks (extend + regression-test the monolith)	~3–5 days (write + register one agent)

The crossover

Approach 1 is faster to the first release by 1–2 weeks. Approach 2 is faster on every release after that. The crossover happens around the third or fourth domain — which, given the payroll roadmap and likely demand for HR analytics, Tipsoi will reach quickly.

4. Cost

Cost has two parts: the one-time engineering build, and the ongoing per-query LLM API spend. The LLM cost is effectively identical between approaches — both send comparable prompts to the same models. The engineering cost differs only in distribution, not total, over the first year.

Ongoing LLM API cost (same for both approaches)

Estimated using current published rates. A typical report query sends a system prompt plus org context (cacheable) and a short user request, and receives a structured response. With prompt caching, the org-context portion costs roughly 90% less on repeat queries.

Model (mid-tier)	Input / Output per 1M tokens	Est. cost per query
Claude Sonnet 4.6	\$3 / \$15	~\$0.02–0.05
GPT-4o	\$2.50 / \$10	~\$0.015–0.04
Gemini Flash (budget)	\$0.30 / \$2.50	~\$0.003–0.01

At, say, 2,000 queries per month across a pilot client, mid-tier spend lands in the rough range of \$40–100/month — small relative to engineering cost. The figures above are directional estimates from published June 2026 rates; actual cost depends on prompt size, caching hit rate, and query volume, and should be confirmed during the pilot.

Cost principle (from the strategy note)

Per-client cost monitoring must be in place from day one. LLM spend scales with usage, so a high-volume client can cost materially more than a light one. This is a metering requirement, not a blocker — the per-query cost is low.

One-time engineering cost

	Approach 1	Approach 2
First pilot (6–8 weeks)	Lower	Slightly higher (foundation)
Each domain after the first	Higher (rework + regression)	Lower (isolated agent)
First-year total (4–6 domains)	Roughly equal — front-loaded	Roughly equal — back-loaded

5. Risk Profile

Risk	Approach 1	Approach 2
Delivery risk for first pilot	Lower — fewer concepts, less to build	Slightly higher — more to stand up first
Long-term maintenance risk	Higher — monolith grows harder to change	Lower — isolated, independently testable agents

Risk	Approach 1	Approach 2
Risk of needing a rewrite	Higher — may hit a structural wall at V3/V4	Very low — the architecture is the end state
Team ramp-up risk	Lower — familiar patterns	Higher — MCP and orchestration are newer
Data safety (both read-only + JWT RBAC)	Equal — same guardrails apply	Equal — same guardrails apply

6. Recommendation

The two approaches are not mutually exclusive in spirit — but they are a real fork in how the first build is structured, and that choice is hard to reverse cheaply later.

Recommended: Approach 2, built lean

Adopt the modular architecture, but scope the first build as tightly as Approach 1 — three agents, one orchestrator, the same 6–8 week pilot. This costs roughly 1–2 extra weeks upfront and buys a system that never needs a rewrite, grows by registration rather than by reworking, and matches the just-in-time, Kaizen operating model. Given the known payroll roadmap and likely demand for more domains, Tipsoi will cross the break-even point quickly.

Choose Approach 1 instead only if: the goal is the fastest possible demo to validate the concept and nothing more, the team has no bandwidth to learn MCP right now, or there is genuine doubt that the system will ever grow beyond a handful of report types. In that case, Approach 1 ships a week or two sooner — with the understanding that scaling it later may require the restructuring that Approach 2 avoids.

Either way, the first pilot is the same bet: one client, Attendance/Leave/Overtime, 6–8 weeks, read-only, human-approved. The decision in this tab is only about what the foundation underneath that pilot looks like.

rough (ignore)

Making Tipsoi APIs Agent-Friendly

The current APIs already provide enough functionality for a strong V1 prototype, because the agent can wrap existing report endpoints and derive insights from them. However, as the agentic layer grows, the APIs should evolve from being primarily UI-driven endpoints into reusable capabilities designed for both humans and AI systems.

Capability-First API Evolution

Going forward, new APIs should be designed with agents as first-class consumers. Instead of exposing endpoints that mirror individual screens, the backend should expose granular, composable capabilities that can be combined by orchestrators and specialist agents.

1. Introduce Aggregation APIs

Many user requests require multiple API calls followed by application-side joins. Examples include:

- Employees with high OT and frequent lateness.
- Employees with low attendance and excessive leave.
- Department-level absenteeism trends.

Rather than forcing agents to orchestrate numerous calls, selected aggregation endpoints should be introduced to encapsulate common cross-domain queries and reduce latency.

2. Add Lookup and Resolution APIs

Agents constantly need to translate natural language into system identifiers. Lightweight APIs should be added for:

- Employee search by name or identifier.
- Department lookup.
- Workplace lookup.
- Shift and shift group resolution.
- Designation resolution.

These endpoints become shared utilities for every future agent.

3. Standardize Filtering

Current APIs expose different parameter structures across modules. Future endpoints should adopt consistent conventions for:

- Date ranges.
- Pagination.
- Sorting.
- Office, department, and employee filters.
- Status values.

Consistency reduces prompt complexity and improves agent reliability.

4. Expose Metadata and Schemas

Agents perform better when they understand what data is available. APIs should expose:

- Field descriptions.

- Enumerations.
- Status code mappings.
- Filter capabilities.
- Supported dimensions and metrics.

This effectively creates self-documenting tools for MCP servers and LLM function calling.

5. Create a Dedicated Analytics Layer

Transactional APIs and analytical APIs have different goals. A separate read-only analytics layer should eventually provide:

- Trends.
- Comparisons.
- Grouped summaries.
- KPI calculations.
- Pre-aggregated statistics.

This reduces orchestration complexity and avoids repeatedly recomputing insights inside the agent.

6. Add Bulk and Batch Endpoints

Agentic systems frequently need to execute several related operations simultaneously. Batch APIs allowing multiple queries in one request can significantly reduce latency and token consumption.

7. Version APIs Around Capabilities

Instead of versioning entire modules, version capabilities independently. Existing APIs remain stable while new agent-friendly capabilities can be introduced without breaking existing UI functionality.

Guiding Principle

Every new backend feature should be built once, exposed as a reusable capability, and consumed equally by the UI, agents, schedulers, and future products.

Under this model, the Tipsoi web application becomes just one client of the platform. The orchestrator, Attendance Agent, Leave Agent, Payroll Agent, Scheduler Agent, and any future AI systems all operate on the same capability layer. This avoids duplicate business logic, simplifies maintenance, and positions Tipsoi for an AI-native future.

Lessons from Orbit Demo

What we learned from Tipsoi AI demo for Orbit

Making the APIs Agent-Ready

The demo proved the agentic layer works on top of the existing Tipsoi report APIs without a single new backend endpoint. That is the right way to validate fast. But the same APIs that made the demo possible are also where most of the friction lived: they were built to serve screens, not agents. This tab lists the concrete tweaks to current endpoints and the new endpoints worth adding so that every agent we ship after this one is cheaper, faster, and more reliable to build.

The principle

Every new backend feature should be built once, exposed as a reusable capability, and consumed equally by the UI, the agents, the scheduler, and any future product. The web app becomes just one client of the platform — not the thing the APIs are shaped around.

1. Tweaks to the APIs We Already Have

None of these require new modules. They are adjustments to endpoints the agent already calls today, aimed at the failure modes we actually hit during the demo.

Current behaviour	The friction it causes for agents	The tweak
Each module exposes its own parameter shape — different date keys, filter names, and status codes across reports.	The orchestrator needs a different mental model per endpoint, which bloats prompts and is the main source of wrong-parameter calls.	Standardise filtering conventions across all report endpoints: one date-range format, one pagination scheme, one sort convention, and shared office/department/employee/status filters.
Dates are accepted only as Unix epoch milliseconds.	The LLM must translate “ <i>last week</i> ” / “ <i>Q2</i> ” into epoch ranges — our highest-frequency failure mode in testing.	Keep epoch support, but also accept ISO dates and named ranges server-side so date logic stops being the agent’s problem. Pair with a shared resolve_date_range utility.
Status and type values are returned as bare integer codes (e.g. status=0 for pending).	The agent has to hard-code mappings it cannot see, so a code change silently breaks answers.	Return human-readable labels alongside codes, and expose the enumerations as metadata (see new endpoints below).
Endpoints return UI-oriented payloads — sometimes pre-formatted, paginated for a table view.	Agents re-parse and re-join data the UI already shaped for rendering, adding latency and token cost.	Offer a clean, fully-typed JSON mode with stable field names. Pagination should be predictable and cursor-driven, not tied to page size of a screen.
Name resolution is implicit — the caller is expected to	Every name-based question forces the agent to fetch and	Back the reports with dedicated lookup endpoints (below) so

Current behaviour	The friction it causes for agents	The tweak
already know employee, office, and shift IDs.	search a list first, multiplying calls.	resolution is one fast, cacheable call.

2. New Endpoints Worth Adding

Ordered roughly by how much each one reduces agent complexity. The first two remove the largest share of the orchestration we currently do in the agent layer.

New capability	What it does	Why agents need it
Lookup & resolution APIs	Employee search by name/ID, department lookup, workplace lookup, shift / shift-group resolution, designation resolution.	Turns “natural language → system identifier” into one cacheable call. Becomes a shared utility every future agent reuses.
Aggregation APIs	Encapsulate common cross-domain queries server-side, e.g. high-OT-and-frequently-late, low-attendance-and-excessive-leave, department absenteeism trends.	Today the agent makes several calls then joins them app-side. One endpoint cuts latency, token use, and a whole class of merge bugs.
Metadata & schema APIs	Expose field descriptions, enumerations, status-code mappings, filter capabilities, and supported dimensions/metrics.	Effectively self-documenting tools for MCP servers and LLM function calling — the agent learns what exists instead of guessing.
Dedicated analytics layer	A separate read-only layer for trends, comparisons, grouped summaries, KPI calculations, and pre-aggregated statistics.	Stops the agent from recomputing the same insights on every query and keeps transactional and analytical concerns apart.
Bulk / batch endpoints	Allow several related queries in a single request.	Agentic flows frequently fire related calls at once; batching meaningfully reduces latency and token consumption.

3. Versioning and Roll-Out

- **Version capabilities, not modules.** Existing UI endpoints stay stable; agent-friendly capabilities are added alongside them without breaking the web app.
- **Additive, never a rewrite.** Each item above can ship independently and be picked up by the orchestrator the moment it exists — no agent rewrite required.
- **RBAC stays inherited.** New endpoints must continue to honour the logged-in user’s JWT so access control comes for free, exactly as it does today.
- **Read-only first.** Every endpoint added for the agent layer is a GET. Write paths remain in the UI behind explicit human action.

Guiding rule for the backend team

If a piece of logic would be useful to the UI, an agent, the scheduler, and a future product, it should be built once as a capability — not re-implemented per consumer. That single decision is what keeps the agentic layer cheap to extend.

The Old Demo vs. The New Approach

Before the architecture described in this dossier, we shipped a working demo of Tipsoi AI to prove the concept. It did its job — it showed the value to stakeholders and surfaced exactly where the rough edges were. This tab records what that demo actually was, how the approach we are now recommending differs, and the lessons that shaped the change. It is here so the reasoning is not lost when the demo material is eventually archived.

1. What the Old System Did

The demo was built on off-the-shelf chatbot tooling (Dify and Botpress) wired to Tipsoi data, fronted by a Flutter mobile chat. It covered a genuinely useful slice of work and demoed well when handled carefully:

- Employee self-service — attendance, late, and leave-balance reports, plus leave and late-regularisation actions with a confirm card before submit.
- Smart notifications — late-arrival and missing-punch alerts that opened straight into a regularisation thread.
- Manager and HR flows — shift roll-ups, attendance drill-downs, employee lookup, and leave approval through the chat with a confirmation gate.
- Knowledge-base Q&A — how-to answers grounded in Tipsoi docs, working in both Bangla and English.

It also carried real demo caveats: one flow per chat for a clean answer, notification timing that was device-dependent, slow loads on long or complex reports, and a “support-ticket” response that was presentational rather than wired to anything. The demo doc was explicit that these had to be rehearsed around.

2. How the New Approach Is Different

Dimension	Old demo	New approach
Foundation	Off-the-shelf chatbot tools (Dify, Botpress) configured for the flows.	A purpose-built modular architecture: one stable orchestrator routing to independent MCP agents.
Growth	Adding a flow meant extending and re-tuning the bot configuration.	Add a domain by registering one agent. The orchestrator never changes.
Data access	Flows wired to data per use case; coupling between what the bot showed and how it fetched it.	Agents wrap the existing report APIs behind a typed MCP contract; data and presentation are separated.
Reliability	Context bleed between flows; one chat could not safely hold multiple flows.	Each agent is isolated and independently testable; date resolution and context are deterministic tools, not guesses.
Safety	Confirm cards in the UI; ticket escalation was shown but not real.	Read-only agents, inherited JWT/RBAC, explicit human approval before any write — enforced architecturally.
Honesty of state	Some capabilities replied as if they did more than they did, for demo feel.	Capabilities are real or absent; the orchestrator routes around what is not built rather than faking it.

3. What We Learned

These are the lessons from building and demoing the first version that directly justify the new direction.

- **Off-the-shelf tooling got us to a demo, not to a product.** It validated the value quickly, but every new flow re-tuned the same configuration and the rough edges compounded. A demo and a maintainable system are different bets.
- **The right architecture beats the fastest one.** Versioned, flow-by-flow growth produces capability cliffs and rework. Building a modular foundation once, then growing by registration, avoids the rebuild the demo path was heading toward.
- **Date and identifier resolution is where agents break.** Translating “last week” or a name into system values was the most common failure. It belongs in deterministic, tested tools and server-side helpers — not in the model’s head. This is the single biggest input to the API tweaks in the previous tab.
- **Don’t fake capability.** The presentational ticket-escalation felt mature in a demo but is a liability in production. The new model only exposes what is real and routes gracefully around what is not.
- **Isolation makes things demoable and debuggable.** The demo needed one-flow-per-chat and careful rehearsal because flows shared context. Independent agents remove that fragility and let different people own different domains.
- **Tooling and infrastructure should be escalated early.** A recurring lesson from the build itself: working around tooling limits costs more than advocating for the right tools up front. The new approach bakes observability and proper interfaces in from day one rather than bolting them on.

The throughline

The demo was the right way to learn cheaply. Everything it taught us — isolate flows, make resolution deterministic, never fake capability, build the foundation once — is exactly what the modular architecture and the agent-ready APIs are designed to deliver.